# Compiler Construction

# Lab 9

**Submitted By:**

Mahnoor Farrukh (52926)

BSCS-6

**Task 1:**

Date

## Task 1:

### Grammer:

$S \to AB$

$A \to aA \mid b$

$B \to cB \mid d$

1) Augmented Grammer:

~~$S' \to AB$~~

$S' \to S$

$S \to AB$

$A \to aA$

$A \to b$

$B \to cB$

$B \to d$

) LR(0) Items:

$I_0$ Closure:

$S' \to \cdot S$

$S \to \cdot AB$

$A \to \cdot aA$

$A \to \cdot b$

GOTO$(I_0, S) \to I_1$

$S' \to S \cdot$

GOTO$(I_0, A) \to I_2$

$S \to A \cdot B$

$B \to \cdot cB$

$B \to \cdot d$

GOTO$(I_0, a) \to I_3$

$A \to a \cdot A$

$A \to \cdot aA$

$A \to \cdot b$

GOTO$(I_0, b) \to I_4$

$A \to b \cdot$

Compute GOTO for $I_2$:

$I_2$ Items:

$S \to A \cdot B$

$B \to \cdot cB$

$B \to \cdot d$

GOTO$(I_2, B) \to I_5$

$S \to AB \cdot$

GOTO$(I_2, c) \to I_6$

$S \to c \cdot B$

$B \to \cdot cB$

$B \to \cdot d$

GOTO$(I_2, d) \to I_7$

$B \to d \cdot$

Compute GOTO for $I_3$:

$I_3$ Items:

$A \to a \cdot A$

$A \to \cdot aA$ (loop)

$A \to \cdot b$ (already exists $I_4$)

Date

GOTO (I3, A) → I8
A → aA·

Compute GOTO for 6
16 items:
B → c·B
B → ·cB (loop)
B → ·d (already exists I7)

GOTO (I6, B) → I9
B → cB·

dot before a → shift
dot at end → reduce
S' comp → acc
GOTO → non-terminals

3) Combined LR (0) Table:

| State | Action | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|
| | a | b | c | d | $ | S | A | B |
| 0 | S3 | S4 | | | | 1 | 2 | |
| 1 | | | | | acc | | | |
| 2 | | | S6 | S7 | | | | 5 |
| 3 | S3 | S4 | | | | | 8 | |
| 4 | r3 | r3 | r3 | r3 | r3 | | | |
| 5 | r1 | r1 | r1 | r1 | r1 | | | |
| 6 | | | S6 | S7 | | | | 9 |
| 7 | r5 | r5 | r5 | r5 | r5 | | | |
| 8 | r2 | r2 | r2 | r2 | r2 | | | |
| 9 | r4 | r4 | r4 | r4 | r4 | | | |

# C++ Code:

Task 2.cpp  Task 1.cpp

```cpp
#include <iostream>
#include <stack>
#include <vector>
#include <map>
using namespace std;

int main() {
    // ====== Action Table ======
    // Format: action[state][terminal] = "sX" (shift), "rY" (reduce), "acc" (accept)
    map<int, map<string, string>> action;

    action[0]["a"] = "s3";
    action[0]["b"] = "s4";

    action[1]["$"] = "acc";

    action[2]["c"] = "s6";
    action[2]["d"] = "s7";

    action[3]["a"] = "s3";
    action[3]["b"] = "s4";

    action[4]["a"] = "r3";
    action[4]["b"] = "r3";
    action[4]["c"] = "r3";
    action[4]["d"] = "r3";
    action[4]["$"] = "r3";

    action[5]["a"] = "r1";
    action[5]["b"] = "r1";
    action[5]["c"] = "r1";
    action[5]["d"] = "r1";
    action[5]["$"] = "r1";

    action[6]["c"] = "s6";
    action[6]["d"] = "s7";

    action[7]["a"] = "r5";
    action[7]["b"] = "r5";
    action[7]["c"] = "r5";
    action[7]["d"] = "r5";
    action[7]["$"] = "r5";

    action[8]["a"] = "r2";
    action[8]["b"] = "r2";
    action[8]["c"] = "r2";
    action[8]["d"] = "r2";
    action[8]["$"] = "r2";

    action[9]["a"] = "r4";
    action[9]["b"] = "r4";
```

```cpp
    action[9]["c"] = "r4";
    action[9]["d"] = "r4";
    action[9]["$"] = "r4";

    // ====== Goto Table ======
    map<int, map<string, int>> goTo;
    goTo[0]["S"] = 1;
    goTo[0]["A"] = 2;
    goTo[2]["B"] = 5;
    goTo[3]["A"] = 8;
    goTo[6]["B"] = 9;

    // ====== Production Lengths (for reduction) ======
    // prodLen[production number] = number of symbols on RHS
    map<int, int> prodLen;
    prodLen[1] = 2; // S → A B
    prodLen[2] = 2; // A → a A
    prodLen[3] = 1; // A → b
    prodLen[4] = 2; // B → c B
    prodLen[5] = 1; // B → d

    // ====== Input ======
    vector<string> input;
    string symbol;
    cout << "Enter input string (space separated, end with $): ";
    while (cin >> symbol) {
        input.push_back(symbol);
        if (symbol == "$") break;
    }

    // ====== Stack for LR(0) Parser ======
    stack<int> st;
    st.push(0); // initial state

    int i = 0; // input pointer

    cout << "\n=== Parsing Steps ===\n";
    while (true) {
        int state = st.top();
        string curr = input[i];

        if (action[state][curr] == "") {
            cout << "Error: Invalid input at symbol " << curr << endl;
            break;
        }

        string act = action[state][curr];

        if (act[0] == 's') { // Shift
            int nextState = stoi(act.substr(1));
            st.push(nextState);
```

```cpp

    int i = 0; // input pointer

    cout << "\n=== Parsing Steps ===\n";
    while (true) {
        int state = st.top();
        string curr = input[i];

        if (action[state][curr] == "") {
            cout << "Error: Invalid input at symbol " << curr << endl;
            break;
        }

        string act = action[state][curr];

        if (act[0] == 's') { // Shift
            int nextState = stoi(act.substr(1));
            st.push(nextState);
            cout << "Shift '" << curr << "', push state " << nextState << endl;
            i++;
        }
        else if (act[0] == 'r') { // Reduce
            int prodNo = stoi(act.substr(1));
            int len = prodLen[prodNo];
            for (int j = 0; j < len; j++) st.pop(); // pop RHS symbols

            int topState = st.top();
            string lhs;

            if (prodNo == 1) lhs = "S";
            else if (prodNo == 2 || prodNo == 3) lhs = "A";
            else lhs = "B";

            st.push(goTo[topState][lhs]); // push goto state

            cout << "Reduce by production " << prodNo << " (" << lhs << " → ";
            if (prodNo == 1) cout << "A B";
            else if (prodNo == 2) cout << "a A";
            else if (prodNo == 3) cout << "b";
            else if (prodNo == 4) cout << "c B";
            else cout << "d";
            cout << "), push state " << st.top() << endl;
        }
        else if (act == "acc") { // Accept
            cout << "Input Accepted \n";
            break;
        }
    }

    return 0;
}
```

# Output:

```cpp
#include <iostream>
#include <stack>
#include <vector>
#include <map>
using namespace std;

int main() {
    // ====== Action Table ======
    // Format: action[state][terminal] = "
    map<int, map<string, string>> action;

    action[0]["a"] = "s3";
    action[0]["b"] = "s4";

    action[1]["$"] = "acc";

    action[2]["c"] = "s6";
    action[2]["d"] = "s7";

    action[3]["a"] = "s3";
    action[3]["b"] = "s4";

    action[4]["a"] = "r3";
    action[4]["b"] = "r3";
    action[4]["c"] = "r3";
    action[4]["d"] = "r3";
    action[4]["$"] = "r3";
```

```
Enter input string (space separated, end with $): a b c d $

=== Parsing Steps ===
Shift 'a', push state 3
Shift 'b', push state 4
Reduce by production 3 (A ? b), push state 8
Reduce by production 2 (A ? a A), push state 2
Shift 'c', push state 6
Shift 'd', push state 7
Reduce by production 5 (B ? d), push state 9
Reduce by production 4 (B ? c B), push state 5
Reduce by production 1 (S ? A B), push state 1
Input Accepted

--------------------------------
Process exited after 8.357 seconds with return value 0
Press any key to continue . . .
```

**Task 2:**

Date

## Task 2:

### Grammer:

$S \to AB$

$A \to aA \mid b$

$B \to cB \mid d$

1) Augmented Grammer:

$S' \to S$

$S \to AB$

$A \to aA$

$A \to b$

$B \to cB$

$B \to d$

First (A) = {a, b}

First (B) = {c, d}

2) Follow Sets:

Follow(S') = {$}

Follow(S) = {$}

Follow(A) = {c, d}

Follow(B) = {$}

3) Canonical LR(0) Item Sets:

Note: SLR uses same LR(0) item sets we computed in last task:

| $I_0$: | $I_1$: | $I_2$: | $I_3$: |
|---|---|---|---|
| $S' \to \cdot S$ | $S' \to S \cdot$ | $S \to A \cdot B$ | $A \to a \cdot A$ |
| $S \to \cdot AB$ | | $B \to \cdot cB$ | $A \to \cdot aA$ |
| $A \to \cdot aA$ | | $B \to \cdot d$ | $A \to \cdot b$ |
| $A \to \cdot b$ | | | |

Date

$I_4$: /
$A \to b.$

$I_5$:
$S \to AB.$

$I_6$:
$B \to c.B$
$B \to .cB$
$B \to .d$

$I_7$:
$B \to d.$

$I_8$:
$A \to aA.$

$I_9$:
$B \to cB.$

4) SLR Action Table:

| State | Action | | | | | GOTO | | |
|---|---|---|---|---|---|---|---|---|
| | a | b | c | d | $ | S | A | B |
| 0 | s3 | s4 | | | | 1 | 2 | |
| 1 | | | | | acc | | | |
| 2 | | | s6 | s7 | | | | 5 |
| 3 | s3 | s4 | | | | | 8 | |
| 4 | | | r4 | r4 | | | | |
| 5 | | | | | r2 | | | |
| 6 | | | s6 | s7 | | | | 9 |
| 7 | | | | | r6 | | | |
| 8 | | | r3 | r3 | | | | |
| 9 | | | | | r5 | | | |

# C++ Code:

Task 2.cpp | Task 1.cpp

```cpp
#include <iostream>
#include <stack>
#include <vector>
#include <map>
using namespace std;

int main(){
    // ====== Action table ======
    // action[state][terminal] = "sX" (shift), "rY" (reduce), "acc" (accept)
    map<int, map<string, string>> action;

    action[0]["a"] = "s3";
    action[0]["b"] = "s4";

    action[1]["$"] = "acc";

    action[2]["c"] = "s6";
    action[2]["d"] = "s7";

    action[3]["a"] = "s3";
    action[3]["b"] = "s4";

    action[4]["c"] = "r4";
    action[4]["d"] = "r4";

    action[5]["$"] = "r2";

    action[6]["c"] = "s6";
    action[6]["d"] = "s7";

    action[7]["$"] = "r6";

    action[8]["c"] = "r3";
    action[8]["d"] = "r3";

    action[9]["$"] = "r5";

    // ====== Goto table ======
    // goTo[state][non-terminal] = next state
    map<int, map<string, int>> goTo;

    goTo[0]["S"] = 1;
    goTo[0]["A"] = 2;
    goTo[2]["B"] = 5;
    goTo[3]["A"] = 8;
    goTo[6]["B"] = 9;

    // ====== Production lengths ======
    // prodLen[production number] = number of symbols on RHS
    map<int, int> prodLen;
    prodLen[1] = 1; // S' → S
```

Task 2.cpp | Task 1.cpp

```cpp
    prodLen[2] = 2; // S → A B
    prodLen[3] = 2; // A → a A
    prodLen[4] = 1; // A → b
    prodLen[5] = 2; // B → c B
    prodLen[6] = 1; // B → d

    // ====== Input ======
    vector<string> input;
    string sym;
    cout << "Enter input string (space separated, end with $): ";
    while (cin >> sym) {
        input.push_back(sym);
        if (sym == "$") break;
    }

    // ====== Stack for SLR Parser ======
    stack<int> st;
    st.push(0); // initial state

    int i = 0; // input pointer

    cout << "\n=== Parsing Steps ===\n";
    while (true) {
        int state = st.top();       // current state
        string curr = input[i];     // current input symbol
        string act = action[state][curr]; // find action from table

        // If no action exists, input is invalid
        if (act == "") {
            cout << "Error: invalid input at " << curr << endl;
            break;
        }

        // ====== Shift ======
        if (act[0] == 's') {
            int next = stoi(act.substr(1)); // get next state number
            st.push(next);                  // push next state
            cout << "Shift '" << curr << "', push state " << next << endl;
            i++; // move to next input symbol
        }

        // ====== Reduce ======
        else if (act[0] == 'r') {
            int p = stoi(act.substr(1)); // get production number
            int len = prodLen[p];        // RHS length of that production

            // pop RHS symbols from stack
            for (int j = 0; j < len; j++)
                st.pop();

            int top = st.top(); // state after popping
```

Task 2.cpp | Task 1.cpp

```cpp
        // ====== Shift ======
        if (act[0] == 's') {
            int next = stoi(act.substr(1)); // get next state number
            st.push(next);                  // push next state
            cout << "Shift '" << curr << "', push state " << next << endl;
            i++; // move to next input symbol
        }

        // ====== Reduce ======
        else if (act[0] == 'r') {
            int p = stoi(act.substr(1)); // get production number
            int len = prodLen[p];        // RHS length of that production

            // pop RHS symbols from stack
            for (int j = 0; j < len; j++)
                st.pop();

            int top = st.top(); // state after popping
            string lhs;

            // decide left-hand side non-terminal
            if (p == 1 || p == 2)
                lhs = "S";
            else if (p == 3 || p == 4)
                lhs = "A";
            else
                lhs = "B";

            // push goto state for LHS
            st.push(goTo[top][lhs]);

            cout << "Reduce by production " << p << " (";
            if (p == 1) cout << "S' -> S";
            else if (p == 2) cout << "S -> A B";
            else if (p == 3) cout << "A -> a A";
            else if (p == 4) cout << "A -> b";
            else if (p == 5) cout << "B -> c B";
            else if (p == 6) cout << "B -> d";
            cout << "), push state " << st.top() << endl;
        }

        // ====== Accept ======
        else if (act == "acc") {
            cout << "Input Accepted\n";
            break;
        }
    }

    return 0;
}
```

# Parse input  a b b $:

Task 2.cpp

```cpp
#include <iostream>
#include <stack>
#include <vector>
#include <map>
using namespace std;

int main(){
    // ====== Action table ======
    // action[state][terminal] = "sX" (shift), "rY" (redu
    map<int, map<string, string>> action;

    action[0]["a"] = "s3";
    action[0]["b"] = "s4";

    action[1]["$"] = "acc";

    action[2]["c"] = "s6";
    action[2]["d"] = "s7";

    action[3]["a"] = "s3";
    action[3]["b"] = "s4";

    action[4]["c"] = "r4";
    action[4]["d"] = "r4";

    action[5]["$"] = "r2";

    action[6]["c"] = "s6";
```

C:\Users\mahno\OneDrive\Dc

```
Enter input string (space separated, end with $): a b b $

=== Parsing Steps ===
Shift 'a', push state 3
Shift 'b', push state 4
Error: invalid input at b

--------------------------------
Process exited after 7.139 seconds with return value 0
Press any key to continue . . .
```

# Parse input  a b c d $:

(globals)

Task 2.cpp

```cpp
#include <iostream>
#include <stack>
#include <vector>
#include <map>
using namespace std;

int main(){
    // ====== Action table ======
    // action[state][terminal] = "sX" (shif
    map<int, map<string, string>> action;

    action[0]["a"] = "s3";
    action[0]["b"] = "s4";

    action[1]["$"] = "acc";

    action[2]["c"] = "s6";
    action[2]["d"] = "s7";

    action[3]["a"] = "s3";
    action[3]["b"] = "s4";

    action[4]["c"] = "r4";
    action[4]["d"] = "r4";

    action[5]["$"] = "r2";

    action[6]["c"] = "s6";
```

Resources | Compile Log | Debug | Find Results | Console

C:\Users\mahno\OneDrive\Dc

```
Enter input string (space separated, end with $): a b c d $

=== Parsing Steps ===
Shift 'a', push state 3
Shift 'b', push state 4
Reduce by production 4 (A -> b), push state 8
Reduce by production 3 (A -> a A), push state 2
Shift 'c', push state 6
Shift 'd', push state 7
Reduce by production 6 (B -> d), push state 9
Reduce by production 5 (B -> c B), push state 5
Reduce by production 2 (S -> A B), push state 1
Input Accepted

--------------------------------
Process exited after 7.206 seconds with return value 0
Press any key to continue . . .
```